Esercitazioni - Design Pattern: Singleton

Molka Gharbaoui

Descrizione del pattern

Descrizione del pattern

- Definisce una classe della quale è possibile la istanziazione di un unico oggetto.
- Il concetto chiave è prevenire la possibilità di creare oggetti di una certa classe tramite il costruttore *new*.
- La classe dovrebbe essere dichiarata ***final*** con l'effetto di impedire la clonazione delle sue istanze.
- Semplice da presentare e implementare in Java, risulta molto utilizzato.

Descrizione del pattern

- Utile nel caso in cui si abbia la necessità di centralizzare informazioni e comportamenti in un’unica entità condivisa da tutti i suoi utilizzatori.
- Conseguenze:
  - accesso controllato alla singola istanza;
  - possibilità di usare un numero variabile di istanze;
  - una maggiore flessibilità

Descrizione del pattern

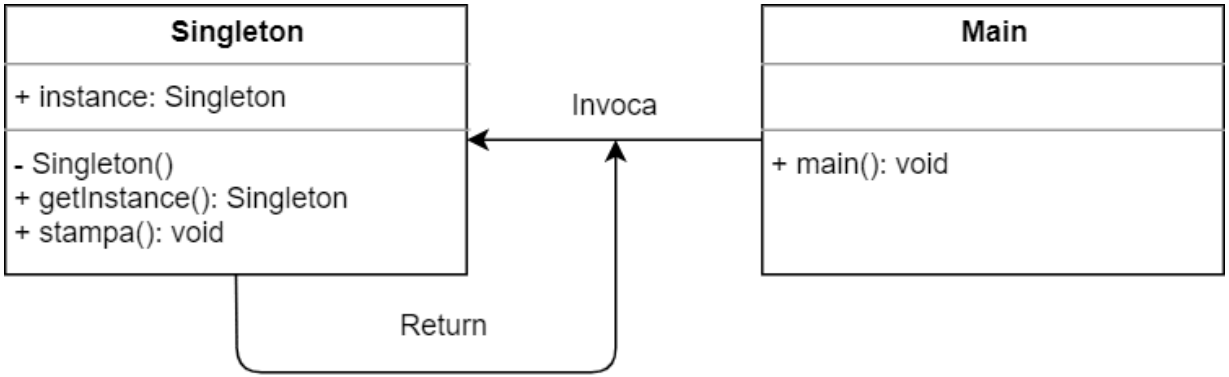
- Vantaggi:
  - Un Singleton può essere scritto rapidamente e facilmente, perché non è popolato da innumerevoli variabili (globali)
  - Incapsula la sua creazione, il che significa che può anche esercitare un controllo preciso su quando e come vi si accede.
  - un singleton viene creato esattamente quando è necessario: caratteristica definita come: «lazy loading»

Descrizione del pattern

- Svantaggi:
  - L'uso disinibito dei singleton porta a uno stato simile a quello della programmazione procedurale
  - Se modificato, non è possibile tracciare quali parti del programma sono interessate, rendendo difficile la manutenzione.
  - Nelle applicazioni con molti utenti, un singleton può ridurre le prestazioni del programma
  - non consente relazioni di ereditarietà.

Struttura del pattern

Struttura del pattern



Struttura del pattern

- Partecipanti:
  - Singleton: definisce un'operazione *Instance* che permette ai client di accedere all'unica istanza disponibile della classe.
  - L'operazione Instance deve essere un'operazione di classe ovvero in Java dovrà essere marcata dalla parola chiave static
  - In una classe Singleton il costruttore è protetto o privato

Struttura del pattern

- Collaborazioni: I client possono accedere ad un’istanza di singleton soltanto invocando l’operazione *getInstance*
- Pattern correlati: altri pattern creazionali potrebbero usare il pattern singleton per esempio per risolvere gli specifici problemi “creazionali” associandoli alla necessità di avere una sola istanza.

Esempi

Esempi

- Esempio 1: Il modo più semplice per implementare una classe singleton richiede due requisiti:
  - Rendere privato il costruttore della classe.
  - Implementare un metodo statico che istanzia e restituisce un oggetto della classe.

Esempi

BasicSingleton.java × MainSingleton.java ×

Source History

```
1
2   public class BasicSingleton {
3
4       private static BasicSingleton instance = null;
5
6       private BasicSingleton(){}
7
8       public static BasicSingleton getInstance(){
9
10          if (instance == null){
11              instance = new BasicSingleton();
12          }
13          return instance;
14      }
15
16      private String value;
17
18      /*Getters and Setters for the variable value*/
19      public String getValue()
20      {return value;}
21
22      public void setValue(String s)
23      {
24       value = s;
25      }
26  }
```

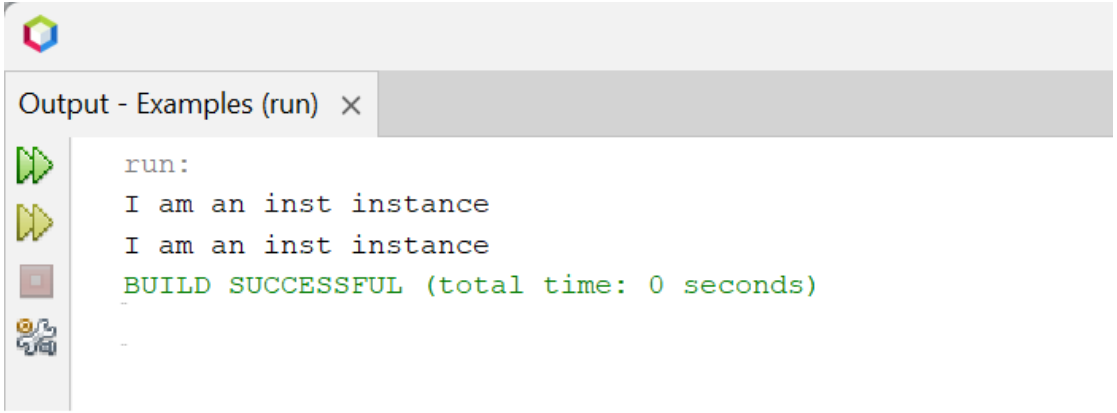
Esempi

BasicSingleton.java × MainSingleton.java ×

Source History

```
1
2   public class MainSingleton {
3
4       public static void main(String[] args){
5
6       BasicSingleton inst = BasicSingleton.getInstance();
7
8       BasicSingleton bar = BasicSingleton.getInstance();
9
10      inst.setValue( s:"I am an inst instance");
11
12      System.out.println( x: inst.getValue());
13
14      System.out.println( x: bar.getValue());
15      }
16  }
17
```

Esempi


Output - Examples (run)
run:
I am an inst instance
I am an inst instance
BUILD SUCCESSFUL (total time: 0 seconds)

Esempi

- Esempio 2: Creare un pool di connessioni per accedere ad un database riutilizzando la stessa connessione per tutti i client.

Esempi

Database.java × DatabaseMain.java ×

Source History

```
1  class Database {
2     private static Database dbObject;
3
4     private Database() {  }
5
6     public static Database getInstance() {
7
8        if(dbObject == null) {
9           dbObject = new Database();}
10
11        return dbObject;
12    }
13
14    public void getConnection() {
15       System.out.println( x: "You are now connected to the database.");
16    }
17 }
18
19
```

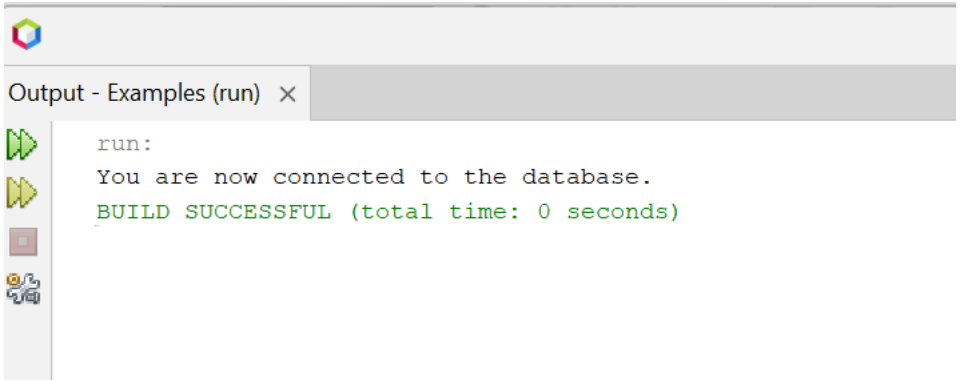
Esempi

Database.java | DatabaseMain.java

Source | History

```
1   class DatabaseMain {
2
3       public static void main(String[] args) {
4           Database db1;
5
6           db1= Database.getInstance();
7
8           db1.getConnection();
9       }
10  }
11
```

Esempi


Output - Examples (run)
run:
You are now connected to the database.
BUILD SUCCESSFUL (total time: 0 seconds)

Esempi

- Esempio 3: Creare una classe Fib che restituisce i valori di una serie di Fibonacci implementando il pattern Singleton

Esempi

Fib.java × TestFib.java ×

Source History

```
1  public class Fib {
2
3    private static Fib obj = new Fib();
4    private int[] x = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233};
5    private int i;
6
7    private Fib() {
8      i = 3;
9    }
10   public static Fib getInstance() {
11     return obj;
12   }
13   public int getValue() {
14     if (i<11) i++;
15     return x[i-1];
16   }
17   public void revert() {
18     i = 0;
19   }
20 }
```

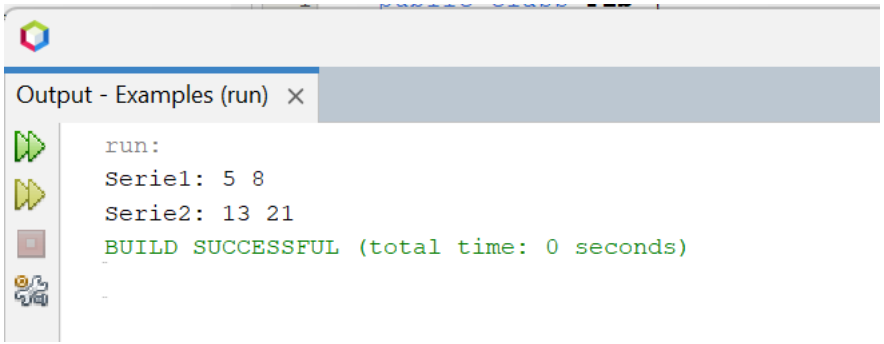
Esempi

Fib.java × TestFib.java ×

Source History

```
1  public class TestFib {
2
3    public static void main(String[] args) {
4
5      Fib f = Fib.getInstance();
6
7      System.out.print("Serie1: "+f.getValue());
8      System.out.println(" "+f.getValue());
9
10
11     Fib f2 = Fib.getInstance();
12
13     System.out.print("Serie2: "+f2.getValue());
14     System.out.println(" "+f2.getValue());
15
16
17   }
18 }
```

Esempi


Output - Examples (run)
run:
Serie1: 5 8
Serie2: 13 21
BUILD SUCCESSFUL (total time: 0 seconds)

Esempi

- Esempio 4: Illustriamo come funziona il pattern Singleton usando l'esempio di una piccola azienda in cui diversi dipendenti utilizzano la stessa stampante.

Esempi

printer.java | Employee.java | MainPrinter.java

Source | History

```
1  public class printer {
2
3          private static printer printer;
4          private int NumberPages;
5          private printer() {
6          }
7          public static printer getInstance() {
8                  if(printer == null)
9                  {   printer = new printer();}
10
11                 return printer;
12         }
13         public void print(String text){
14                 System.out.println(text +
15                                 "\n" + "number of pages printed today" + ++ NumberPages +
16                                 "\n" + "---------");
17         }
18 }
19
```

Esempi

printer.java | Employee.java | MainPrinter.java

Source | History

```
1  public class Employee {
2
3          private final String name;
4          private final String position;
5          private final String role;
6
7          public Employee(String name, String position, String role) {
8                  this.name = name;
9                  this.position = position;
10                 this.role = role;
11         }
12
13         public void printCurrentRole (){
14
15                 printer printer = null;
16                 printer = printer.getInstance();
17                 printer.print("employee: " + name + "\n" +
18                         "Position: " + position + "\n" +
19                         "Role: " + role + "\n");
20         }
21 }
```

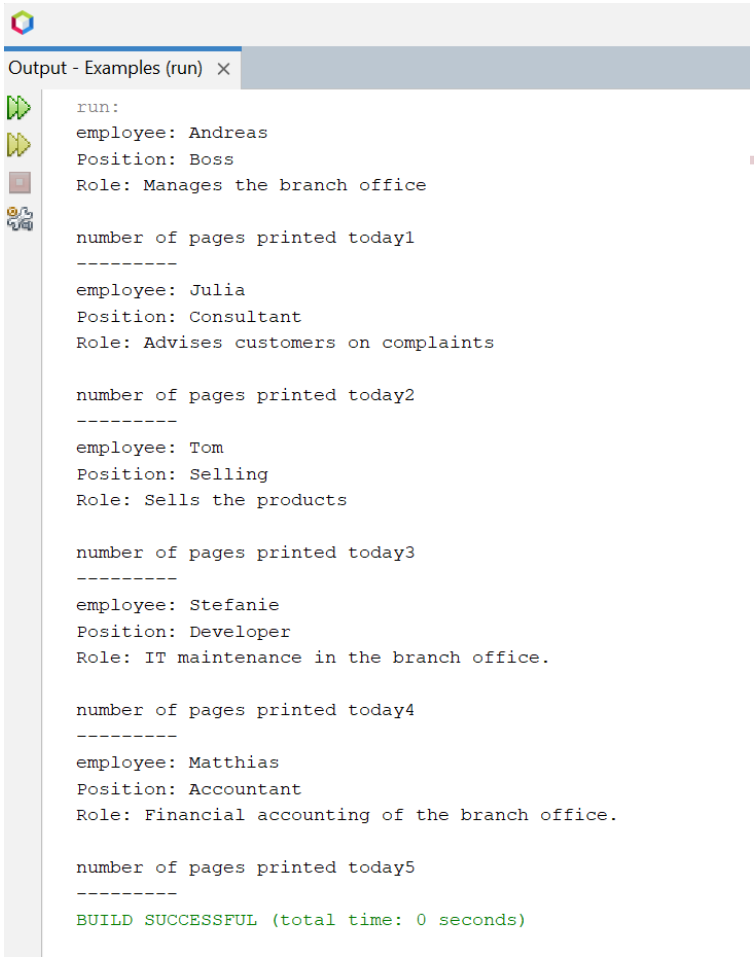
Esempi

printer.java × Employee.java × MainPrinter.java ×

Source History

```
1  public class MainPrinter {
2
3      public static void main(String[] args) {
4          Employee andreas = new Employee (name: "Andreas",
5                  position: "Boss",
6                  role: "Manages the branch office");
7          Employee julia = new Employee (name: "Julia",
8                  position: "Consultant",
9                  role: "Advises customers on complaints");
10         Employee tom = new Employee (name: "Tom",
11                 position: "Selling",
12                 role: "Sells the products");
13         Employee stefanie = new Employee (name: "Stefanie",
14                 position: "Developer",
15                 role: "IT maintenance in the branch office.");
16         Employee matthias = new Employee (name: "Matthias",
17                 position: "Accountant",
18                 role: "Financial accounting of the branch office.");
19         andreas.printCurrentRole();
20         julia.printCurrentRole ();
21         tom.printCurrentRole ();
22         stefanie.printCurrentRole ();
23         matthias.printCurrentRole ();
24     }
25 }
```

Esempi


Output - Examples (run)
run:
employee: Andreas
Position: Boss
Role: Manages the branch office
number of pages printed today1
---
employee: Julia
Position: Consultant
Role: Advises customers on complaints
number of pages printed today2
---
employee: Tom
Position: Selling
Role: Sells the products
number of pages printed today3
---
employee: Stefanie
Position: Developer
Role: IT maintenance in the branch office.
number of pages printed today4
---
employee: Matthias
Position: Accountant
Role: Financial accounting of the branch office.
number of pages printed today5

BUILD SUCCESSFUL (total time: 0 seconds)